

Author	Reviewer	Approver
BaoHT5		

Workshop 1: Building Real-World AI-Powered Apps Using Python and Azure OpenAI API

Workshop Objective

Provide participants with hands-on experience building practical AI-powered applications using Python and the Azure OpenAI API, focusing on automating repetitive tasks via the official Azure OpenAI Python library. Each mentee will develop their own AI-powered tool.

Open-ended Question

In this workshop, you will design and implement an AI-powered application that automates a repetitive task relevant to your own needs or interests by leveraging Python and Azure OpenAI API. You will learn how to integrate Azure OpenAI's API with Python code, handle prompts effectively, and build a useful assistant for tasks like text generation, summarization, or automation.

Applications (Choose One or Define Your Own)

- Email Draft Generator
- Meeting Notes Summarizer
- Code Comment Generator
- Data Entry Automation
- FAQ Answering Bot

AI Tools Usage Guidelines

Tool	Usage
Azure OpenAI Python SDK (openai lib)	Connect to ChatGPT models, send prompts, and process completions
Azure OpenAI API	Craft prompts for generating, summarizing, or transforming text

Agenda

- 1. Introduction to Azure OpenAI API and Python Integration**
Overview of Azure OpenAI API, environment setup, installing Azure OpenAI Python SDK, API key management, and basic ChatGPT API calls.
- 2. Prompt Engineering and Task Definition**
Define the repetitive task to automate. Draft prompts tailored for ChatGPT to solve or assist with that task.
- 3. Building the AI App Prototype**
Implement Python scripts that connect to Azure OpenAI API, send crafted prompts, and handle responses. Implement basic input/output UI (CLI or minimal GUI).
- 4. Testing and Iteration**
Test your AI assistant with sample inputs. Refine prompts and code to improve output quality and reliability.
- 5. Demo and Sharing**
Each group/ team presents their AI-powered app, shares challenges, and discusses improvements.

Deliverables by End of Workshop

- A working Python script/application that uses Azure OpenAI API to automate a chosen repetitive task.
- Well-crafted prompt templates embedded in the code.
- Basic user guide or instructions on using the AI assistant.
- Short demo of the application in action.

Example Project

Example Topic: AI-Powered Meeting Notes Summarizer

Project Brief

Create a Python tool that takes raw meeting transcript text and produces concise meeting notes using ChatGPT via Azure OpenAI API.

Example Main Features

- Input raw transcript text file or pasted text
- Send prompt to ChatGPT to summarize key points
- Output summary as text file or console print
- Optionally, highlight action items

Sample Code Snippet (Backend Developer Style)

```
from openai import AzureOpenAI
import os

client = AzureOpenAI(
    api_version="2024-07-01-preview",
    azure_endpoint=os.getenv("AZURE_OPENAI_ENDPOINT"),
    api_key=os.getenv("AZURE_OPENAI_API_KEY"),
)

def summarize_meeting(text):
    prompt = f"Summarize the following meeting transcript into concise notes:\n\n{text}"
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": prompt}],
        max_tokens=300,
        temperature=0.3
    )
    return response.choices[0].message['content']

if __name__ == "__main__":
    with open("meeting_transcript.txt") as f:
        transcript = f.read()
        notes = summarize_meeting(transcript)
        print("Meeting Notes:\n", notes)
```

1. Task Definition & Prompt Examples

Task:	Automate	creation	of	meeting	notes	from	transcripts.
Prompt							example:

"Please summarize the key points and action items from the following meeting transcript: [transcript text]."

2. MVP Feature List

- Load input text from file or user input
 - Generate summary using Azure OpenAI ChatCompletion API
 - Display or save the summarized notes
 - Handle simple errors and empty inputs
-

3. Basic System Design (Conceptual)

- Input Module (Read text input)
 - Azure OpenAI API Wrapper (send prompt, receive response)
 - Output Module (display or save result)
 - CLI or basic UI for user interaction
-

4. Testing & Refinement

- Test with different transcript styles (formal, informal)
 - Adjust prompt for length, style, detail level
 - Handle API errors or rate limits gracefully
-

5. Example Test Cases

Test Case ID	Description
TC_01	Summarize a short meeting transcript → concise notes produced
TC_02	Summarize a long transcript with multiple topics → clear sectioned notes
TC_03	Empty input text → returns helpful error or prompt to retry
TC_04	Invalid API key → handle authentication error